

Исключения и их обработка

Докладчик: Евграфов Михаил



Что такое исключения

Исключительные ситуация

```
def calculate_average(numbers: list[int]) -> float:
    return sum(numbers) / len(numbers)


print(calculate_average([4, 8, 15]))
# 9.0
print(calculate_average([]))
# ...
# ZeroDivisionError: division by zero
```

Трассировка

```
Traceback (most recent call last):
  File "example.py", line 6, in <module>
    print(calculate_average([]))
          ~~~~~~~~~~~~~~~~~~^^^^
  File "example.py", line 2, in calculate_average
    return sum(numbers) / len(numbers)
           ~~~~~~~~~~~~~^~~~~~~~~~~~~~
ZeroDivisionError: division by zero
```

заголовок

кадры стека

заголовок

Объект исключения

```
try:
    calculate_average([])

except ZeroDivisionError as error:
    print(type(error))                  # <class 'ZeroDivisionError'>
    print(error)                        # division by zero
    print(error.args)                   # ('division by zero',)
    print(error.__traceback__)          # <traceback object at 0x...>
```

Сигнальные значения

```
def parse_age(raw_age: str) -> int:
    if not raw_age.isdigit():
        return -1

    return int(raw_age)

age = parse_age("abc")
print(age + 1)
# 0 - ошибка осталась незамеченной
```

Недостатки сигнальных значений

признак ошибки совпадает по типу с результатом

ошибку легко проигнорировать

нет информации о причине

признак ошибки нужно передавать вручную

Исключение

```
def parse_age(raw_age: str) -> int:
    return int(raw_age)


age = parse_age("abc")
# ValueError: invalid literal for int() ...
# не выполнится
print(age + 1)
```

Преимущества исключений

ошибку нельзя не заметить

результат функции всегда корректен

сохранение данных о причине

автоматическое распространение

обнаружение и обработка разделены

SyntaxError

```
print("Hello"
# SyntaxError: '(' was never closed
```

Ошибки и исключения

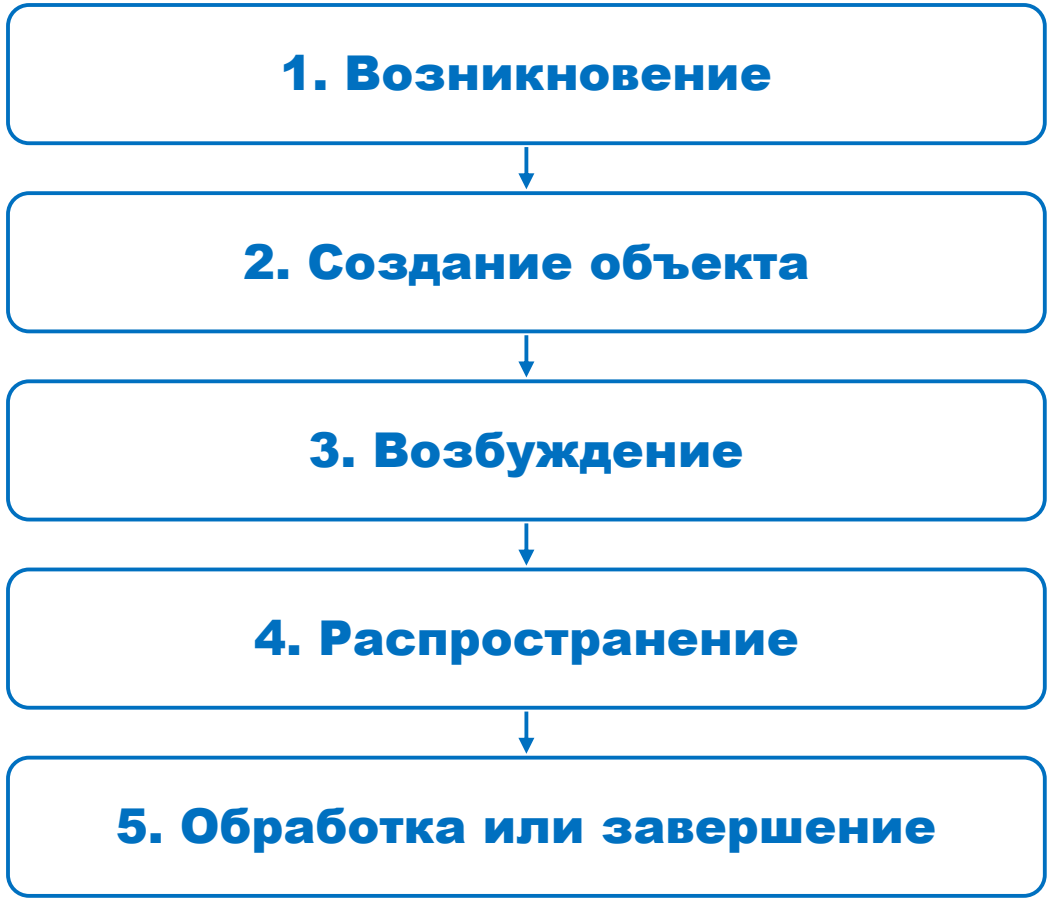
до выполнения

`SyntaxError`
обработать нельзя

во время выполнения

исключения
обработать можно

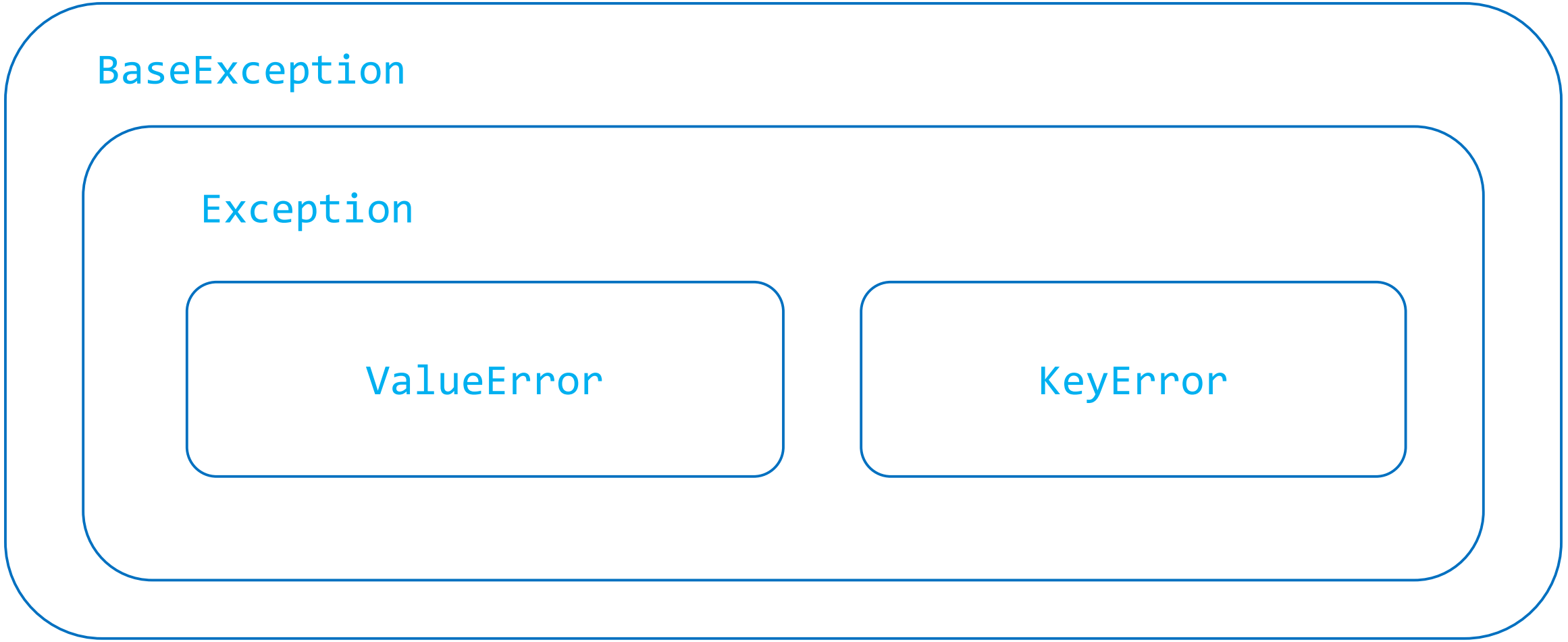
Жизненный цикл исключения



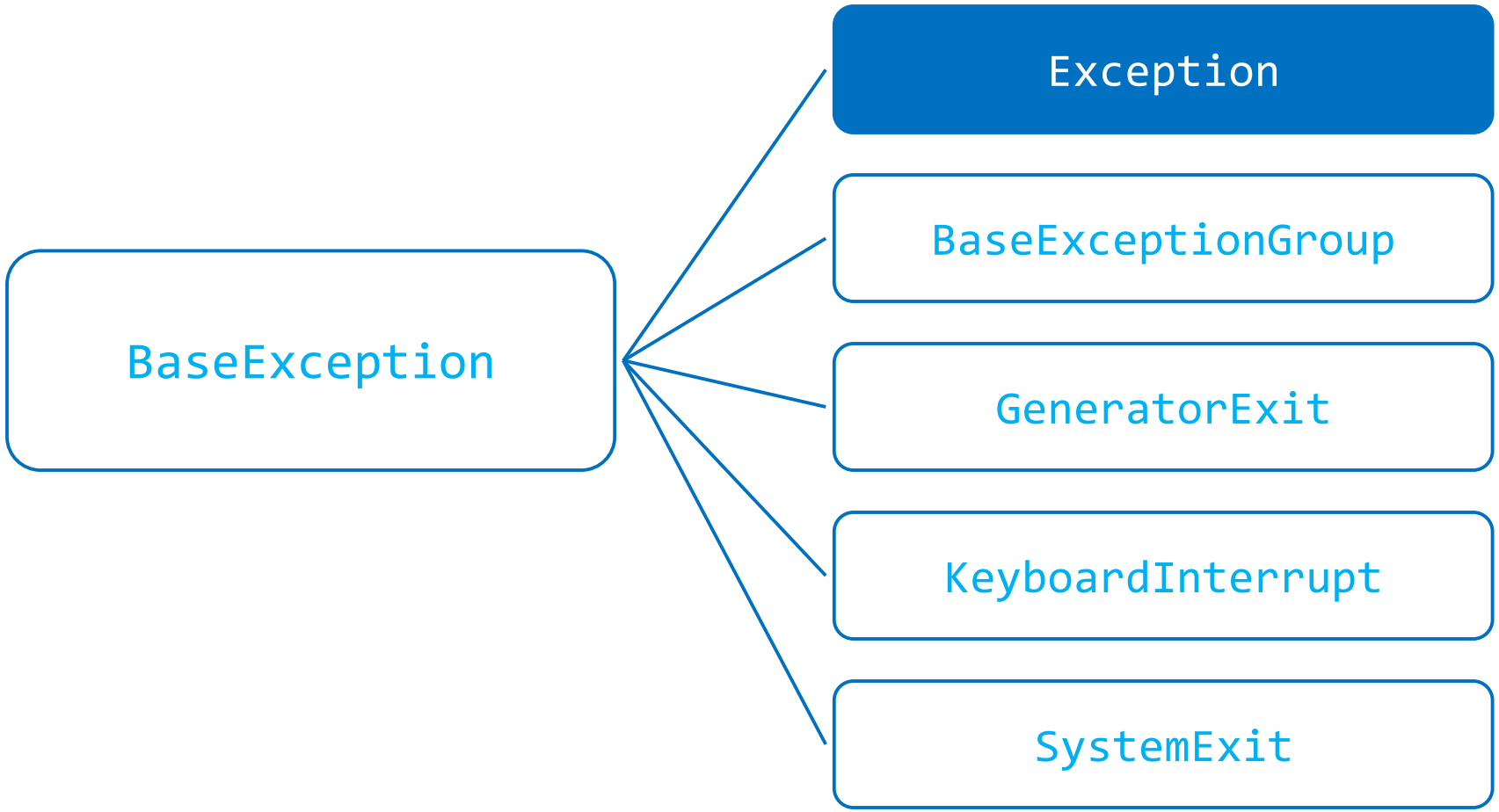


Иерархия исключений

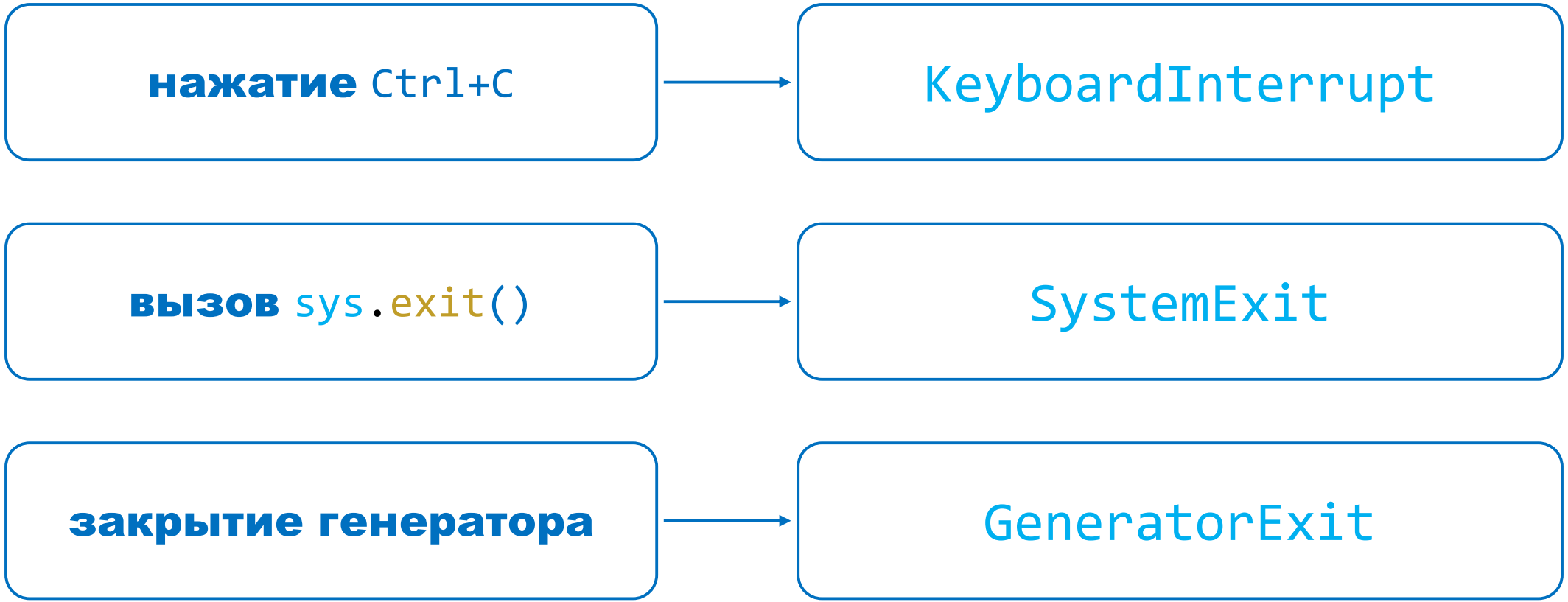
Вложенность типов



Верхний уровень



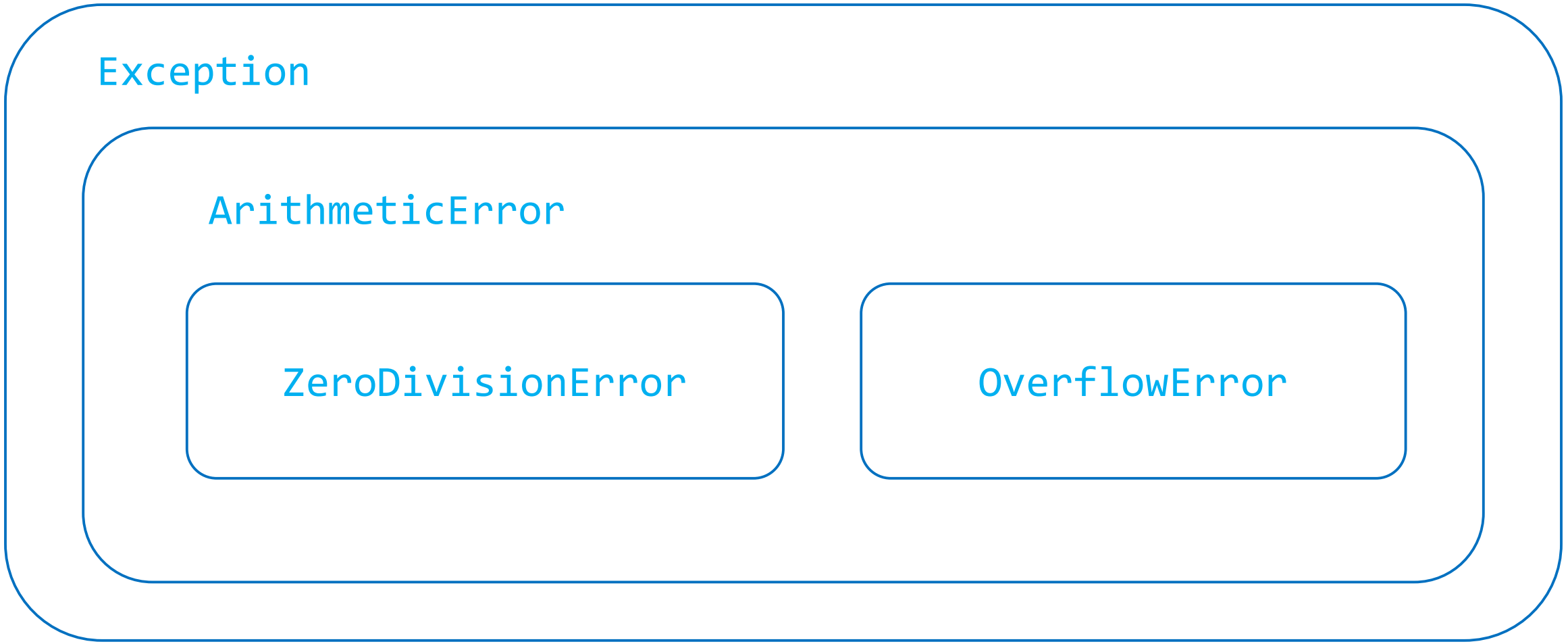
Особенные исключения



Иерархия Exception

```
Exception
  ├── ArithmeticError
  │     ├── ZeroDivisionError
  │     └── OverflowError
  ├── LookupError
  │     ├── IndexError
  │     └── KeyError
  ├── OSError
  │     ├── FileNotFoundError
  │     ├── PermissionError
  │     └── FileExistsError
  ├── ValueError
  ├── TypeError
  ├── AttributeError
  ├── NameError
  └── ...
```

Иерархия Exception





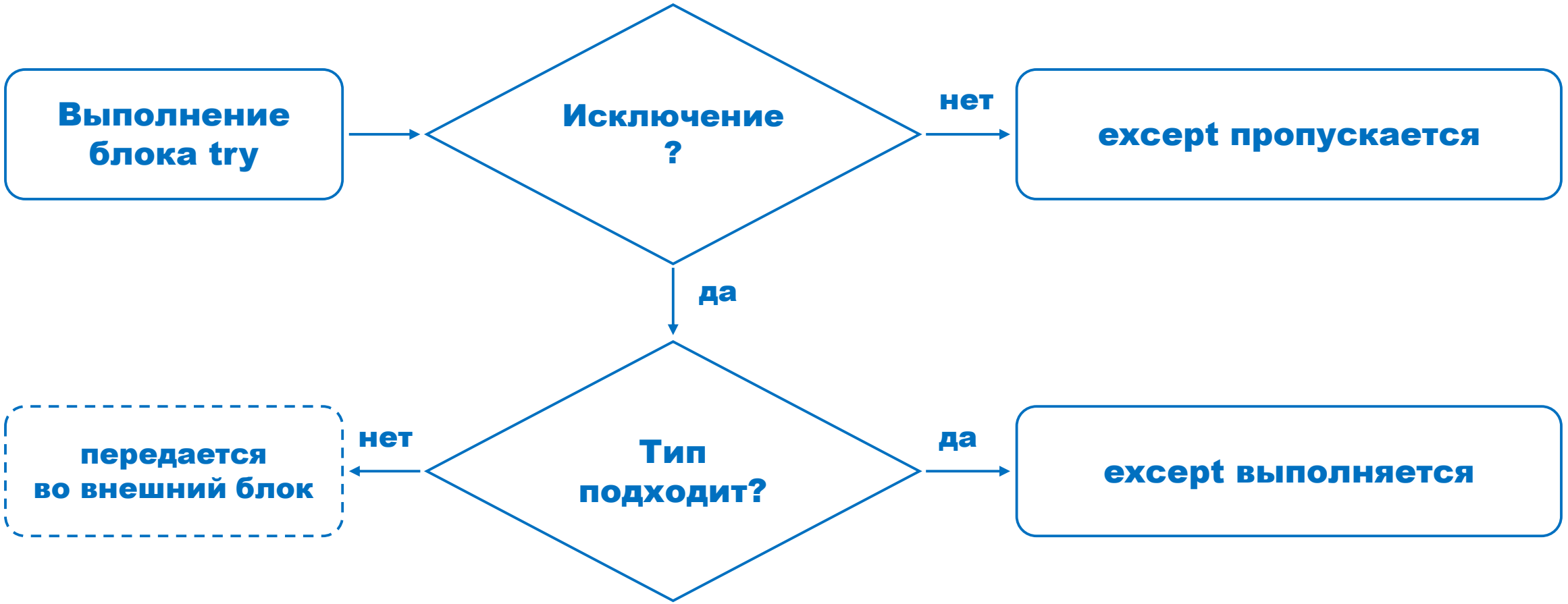
try-except

Синтаксис try-except

```
try:
    # код, который может возбудить исключение
    ...

except ExceptionType:
    # код обработки исключения типа ExceptionType
    ...
```

Порядок выполнения



Выполнение блока except

```
raw_age = "abc"

try:
    age = int(raw_age)     # int("abc") - ошибка
    print(f"Age: {age}")   # не будет выполнено

except ValueError:
    print(f"Invalid age: {raw_age!r}")
    # Invalid age: 'abc'

print("Program continues")
# Program continues
```

Блок except не выполнен

```
try:
    result = 10 / 0  # ошибка ZeroDivisionError

except ValueError:   # тип не подходит
    print("Invalid value")

# ZeroDivisionError: division by zero
```

Привязка исключения as

```
try:
    # код, который может возбудить исключение
    ...

except ExceptionType as error:
    # переменная error ссылается на объект исключения
    ...
```

Пример привязки

```
try:
    int("abc")

except ValueError as error:
    print(f"Error type: {type(error).__name__}")
    # Error type: ValueError
    print(f"Error message: {error}")
    # Error message: invalid literal for int()...
```

Время жизни error

```
try:
    int("abc")

except ValueError as error:
    pass

print(error)
# NameError: name 'error' is not defined
```

Блок except обязателен

```
try:
    age = int("42")

print(age)
# SyntaxError: expected 'except' or 'finally' block
```

Голый except

```
try:
    age = int(raw_age)

except:
    print("Something went wrong")
```

Проблемы голого except

нельзя остановить программу

скрываются ошибки в коде

нет доступа к исключению

Программу не остановить

```
while True:
    try:
        command = input("Enter a command: ")

    except:
        print("Something went wrong")
```

Скрытые ошибки

```
raw_age = "42"

try:
    # опечатка: raw_ge вместо raw_age
    age = int(raw_ge)

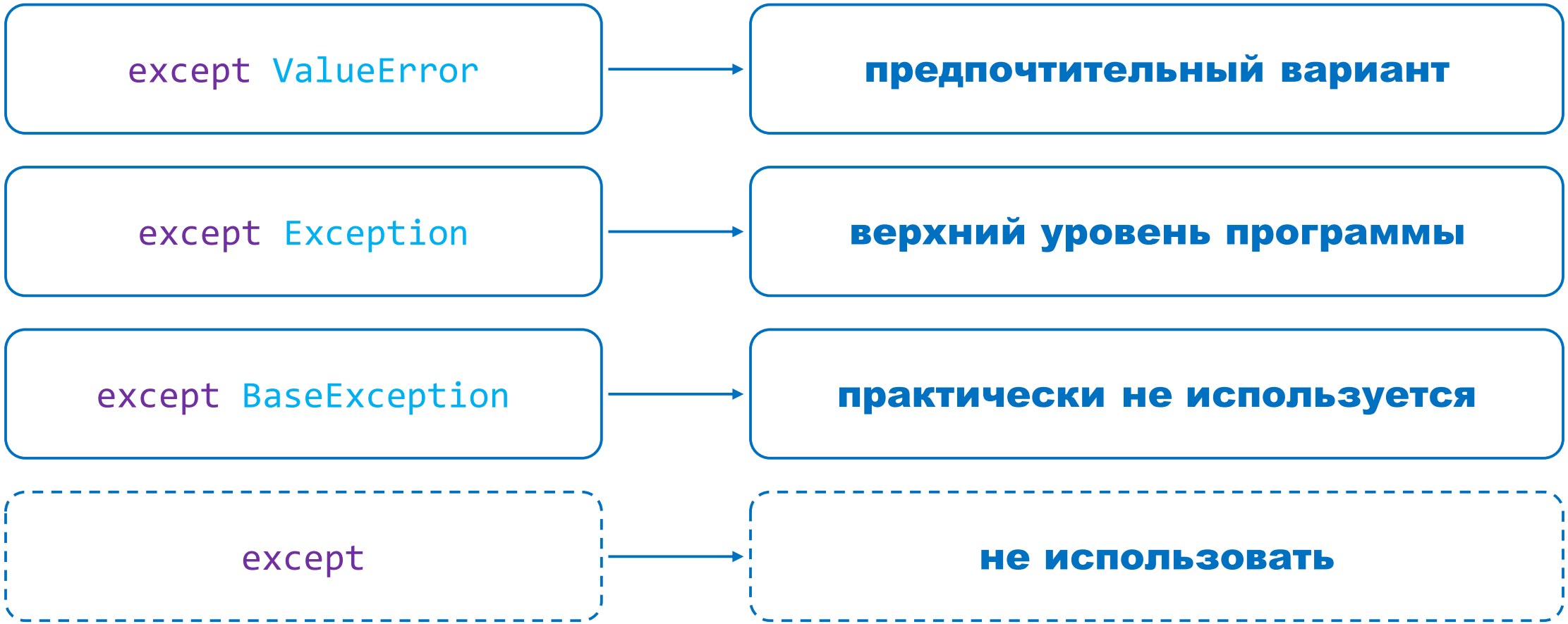
except:
    print("Invalid age")
    # Invalid age
```

except Exception

```
try:
    process_data()

except Exception as error:
    print(f"Unexpected error: {error}")
```

Что перехватывает except



Широкий try

```
def process_age(raw_age: str) -> None:
    try:
        age = int(raw_age)
        discount = calculate_discount(age)
        save_discount(discount)

    except ValueError:
    # ValueError может прийти из любой из трех строк
        print(f"Invalid age: {raw_age!r}")
```

Узкий try

```
def process_age(raw_age: str) -> None:
    try:
        age = int(raw_age)

    except ValueError:
        print(f"Invalid age: {raw_age!r}")
        return

    discount = calculate_discount(age)
    save_discount(discount)
```

Несколько блоков except

```
def divide_numbers(raw_dividend: str, raw_divisor: str) -> None:
    try:
        result = int(raw_dividend) / int(raw_divisor)
        print(f"Result: {result}")

    except ValueError:
        print("Both values must be integers")

    except ZeroDivisionError:
        print("Divisor must not be zero")

divide_numbers("10", "2")    # Result: 5.0
divide_numbers("10", "abc")  # Both values must be integers
divide_numbers("10", "0")    # Divisor must not be zero
```

Общий except

```
def parse_ratio(raw_dividend: str, raw_divisor: str) -> float:
    try:
        return int(raw_dividend) / int(raw_divisor)

    except (ValueError, ZeroDivisionError) as error:
        print(f"Cannot calculate ratio: {error}")
        return 0.0

print(parse_ratio("10", "4"))    # 2.5
print(parse_ratio("10", "abc"))
# Cannot calculate ratio: invalid literal ...
print(parse_ratio("10", "0"))
# Cannot calculate ratio: division by zero
```

Комбинация блоков

```
try:
    result = int(raw_dividend) / int(raw_divisor)

except (ValueError, TypeError):
    print("Invalid input")

except ZeroDivisionError:
    print("Divisor must not be zero")
```

Порядок поиска блока

```
try:
    int("abc")                          # возбуждается ValueError

except KeyError:                        # не подходит
    print("KeyError block")

except ValueError:                      # подходит - выполняется
    print("ValueError block")
    # ValueError block

except Exception:
    # подошел бы, но пропускается: выше есть подходящий блок
    print("Exception block")
```

Общий блок перед частным

```
try:
    int("abc")

except Exception:
    print("Generic error")
    # Generic error

except ValueError:
    # недостижимый код
    print("Invalid value")
```

Исправление ошибки

```
try:
    int("abc")

except ValueError:
    print("Invalid value")
    # Invalid value

except Exception:
    print("Generic error")
```

Независимые обработчики

```
try:
    result = 10 / 0

except ValueError:
    print("Invalid value")

except ZeroDivisionError:
    print("Divisor must not be zero")
    # Divisor must not be zero
```

Блок else

```
try:
    # код, который может возбудить ожидаемое исключение
    ...

except ExceptionType:
    # обработка исключения
    ...

else:
    # выполняется, только если исключения в try не было
    ...
```

Пример

```
def parse_age(raw_age: str) -> None:
    try:
        age = int(raw_age)

    except ValueError:
        print("Invalid age")

    else:
        print(f"Age is {age}")

parse_age("42")   # Age is 42
parse_age("abc")  # Invalid age
```

Порядок выполнения

ситуация	try	except	else
исключения нет	выполнен целиком	пропущен	**выполняется**
исключение перехвачено	прерван	**выполняется**	пропущен
исключение не перехвачено	прерван	пропущен	пропущен

Все в try

```
def calculate_discount(age: int) -> float:
    return float("ten percent")  # ValueError

try:
    age = int("42")
    # ValueError возникнет здесь
    discount = calculate_discount(age)

except ValueError:
    print("Invalid age")
    # Invalid age
```

Использование else

```
try:
    age = int("42")

except ValueError:
    print("Invalid age")

else:
    discount = calculate_discount(age)
    # ValueError не перехватывается
```

Когда else не нужен

```
try:
    age = int(raw_age)

except ValueError:
    age = 0

print(f"Age is {age}")
```



Инструкция raise

Явное возбуждение

```
def calculate_square_root(number: float) -> float:
    if number < 0:
        raise ValueError("number must not be negative")

    return number ** 0.5

print(calculate_square_root(16))
# 4.0
print(calculate_square_root(-4))
# ValueError: number must not be negative
```

Что делает raise



Способы создания

```
# исключение с сообщением
raise ValueError("number must not be negative")

# только тип: сообщение пустое
raise ValueError

# эквивалентная запись
raise ValueError()
```

Атрибут args

```
try:
    raise ValueError("first", 42)

except ValueError as error:
    print(error.args)  # ('first', 42)
    print(error)       # ('first', 42)
```

Повторное возбуждение

```
def load_age(raw_age: str) -> int:
    try:
        return int(raw_age)
    except ValueError:
        print("Failed to parse age")
        raise

try:
    load_age("abc")
    # Failed to parse age
except ValueError as error:
    print(f"Caught outside: {error}")
    # Caught outside: invalid literal...
```

Когда нужен re-raise

запись в журнал ошибка не подавляется

освобождение ресурсов откат изменений

условная обработка по атрибуту исключения

Условная обработка

```
import errno

try:
    process_data()

except OSError as error:
    if error.errno != errno.ENOSPC:
        raise

    print("Disk is full, clearing cache")
```

Новое исключение

```
def load_age(raw_age: str) -> int:
    try:
        return int(raw_age)

    except ValueError:
        raise RuntimeError("failed to load age")

try:
    load_age("abc")

except RuntimeError as error:
    print(f"Caught: {error}")
    # Caught: failed to load age
```

Неявная цепочка исключений

```
def load_age(raw_age: str) -> int:
    try:
        return int(raw_age)

    except ValueError:
        raise RuntimeError("failed to load age")


load_age("abc")
```

Трассировка

```
Traceback (most recent call last):
  File "example.py", line 3, in load_age
    return int(raw_age)
ValueError: invalid literal for int() with base 10: 'abc'

During handling of the above exception, another exception occurred:

Traceback (most recent call last):
  File "example.py", line 9, in <module>
    load_age("abc")
  File "example.py", line 6, in load_age
    raise RuntimeError("failed to load age")
RuntimeError: failed to load age
```

Явная цепочка исключений

```
def load_age(raw_age: str) -> int:
    try:
        return int(raw_age)

    except ValueError as error:
        error_msg = "failed to load age"
        raise RuntimeError(error_msg) from error


load_age("abc")
```

Трассировка

```
Traceback (most recent call last):
  File "example.py", line 3, in load_age
    return int(raw_age)
ValueError: invalid literal for int() with base 10: 'abc'

The above exception was the direct cause of the following exception:

Traceback (most recent call last):
  File "example.py", line 9, in <module>
    load_age("abc")
  File "example.py", line 6, in load_age
    raise RuntimeError("failed to load age") from error
RuntimeError: failed to load age
```

Два вида связи

Неявная	Явная
`raise NewError(...)`	`raise NewError(...) from error`
During handling of the above exception, another exception occurred	The above exception was the direct cause of the following exception
возникло в процессе обработки	**первое — прямая причина второго**

raise from None

```
def get_price(prices: dict[str, int], product: str) -> int:
    try:
        return prices[product]

    except KeyError:
        error_msg = f"unknown product: {product}"
        raise KeyError(error_msg) from None
```

```
Traceback (most recent call last):
  File "example.py", line 11, in <module>
    get_price({"apple": 10}, "pear")
  File "example.py", line 6, in get_price
    raise KeyError(f"unknown product: {product}") from None
KeyError: 'unknown product: pear'
```

Когда применять from None

применять	не применять
исходное исключение — деталь реализации	исходное исключение помогает найти причину
новое исключение полностью описывает проблему	подавление цепочки скрывает важную информацию

Выбор формы возбуждения

передать то же исключение дальше	`raise`
заменить, сохранив причину	`raise NewError(...) from error`
заменить, скрыв причину	`raise NewError(...) from None`
возбудить без явной связи	`raise NewError(...)`



Распространение исключений

Код программы

```
def read_age(raw_age: str) -> int:
    return int(raw_age)

def build_profile(raw_age: str) -> dict[str, int]:
    return {"age": read_age(raw_age)}

def main() -> None:
    profile = build_profile("abc")
    print(profile)

main()
```

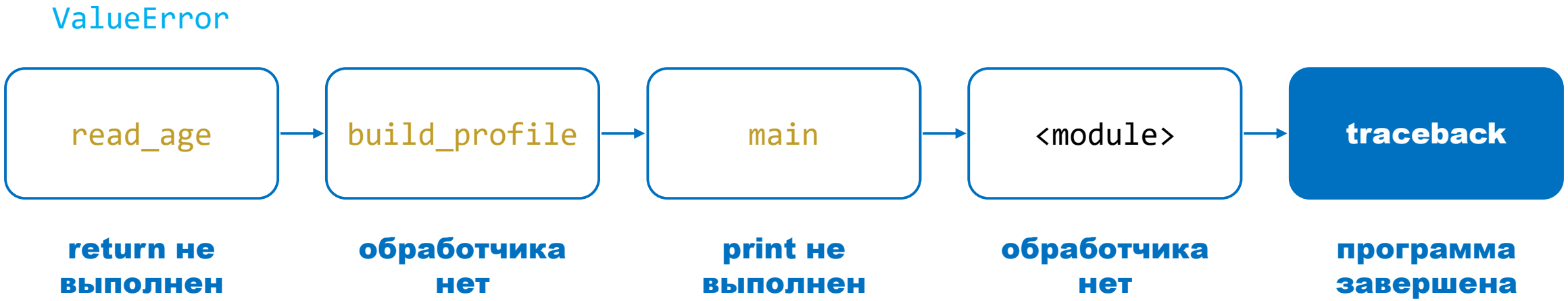
Стек вызовов

```
read_age("abc")          <- здесь возникает ValueError
build_profile("abc")
main()
<module>
```

Результат выполнения

```
Traceback (most recent call last):
  File "example.py", line 14, in <module>
    main()
  File "example.py", line 10, in main
    profile = build_profile("abc")
  File "example.py", line 6, in build_profile
    return {"age": read_age(raw_age)}
  File "example.py", line 2, in read_age
    return int(raw_age)
ValueError: invalid literal for int() with base 10: 'abc'
```

Подъем по стеку вызовов



Перехват

```
def read_age(raw_age: str) -> int:
    return int(raw_age)

def build_profile(raw_age: str) -> dict[str, int]:
    try:
        return {"age": read_age(raw_age)}
    except ValueError:
        return {"age": 0}

def main() -> None:
    profile = build_profile("abc")
    print(profile)

main()
# {'age': 0}
```

Выбор ближайшего обработчика

```
def read_age(raw_age: str) -> int:
    try:
        return int(raw_age)
    except ValueError:
        print("read_age: handled")
        return 0

def main() -> None:
    try:
        print(read_age("abc"))
    except ValueError:
        print("main: handled")

main()
# read_age: handled
# 0
```

Выбор ближайшего обработчика

```
def read_age(raw_age: str) -> int:
    try:
        return int(raw_age)
    except KeyError:  # не подходит для ValueError
        print("read_age: handled")
        return 0

def main() -> None:
    try:
        print(read_age("abc"))
    except ValueError:
        print("main: handled")

main()
# main: handled
```

Правила обработки

обрабатывать там, где известно, что делать

не перехватывать без необходимости

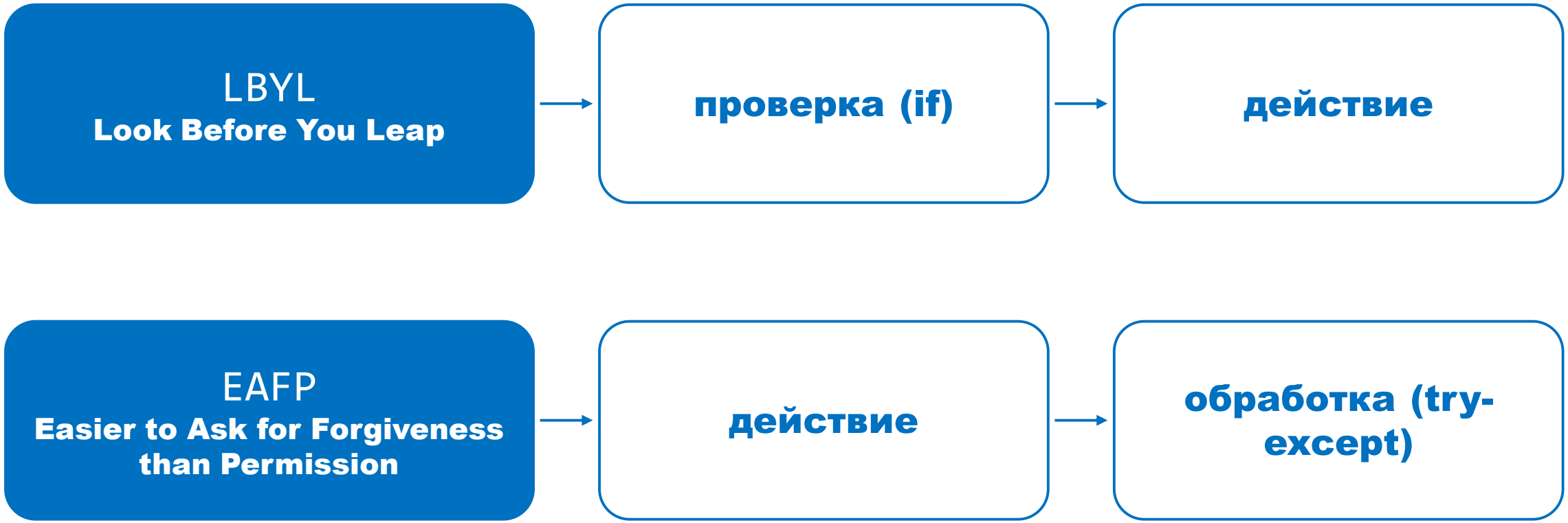
общий обработчик — на верхнем уровне

частичная обработка — повторный raise



LBYL и EAFP

Два подхода


LBYL
Look Before You Leap
проверка (if)
действие
EAFP
Easier to Ask for Forgiveness than Permission
действие
обработка (try-except)

LBYL и EAFP в коде

LBYL

```
def get_price_lbyl(prices: dict[str, int], product: str) -> int:
    if product in prices:
        return prices[product]
    return 0
```

EAFP

```
def get_price_eafp(prices: dict[str, int], product: str) -> int:
    try:
        return prices[product]
    except KeyError:
        return 0
```

Недостатки LBYL

гонки (TOCTOU)

неполная проверка

двойная работа

LBYL и гонки

```
import os

path = "data.txt"

if os.path.exists(path):
    # здесь другой процесс может удалить файл
    with open(path) as file:
        # возможный FileNotFoundError
        ...
```

EAFP без гонки

```
try:
    with open(path) as file:
        content = file.read()

except FileNotFoundError:
    content = ""
```

Неполные проверки

```
def parse_number_lbyl(text: str) -> int | None:
    if text.isdigit():
        return int(text)

    return None

print(parse_number_lbyl("42"))
# 42
print(parse_number_lbyl("-5"))
# None - допустимое число отвергнуто
print(parse_number_lbyl(" 7 "))
# None - int() допускает пробелы по краям
```

Исправление через EAFP

```
def parse_number_eafp(text: str) -> int | None:
    try:
        return int(text)

    except ValueError:
        return None

print(parse_number_eafp("-5"))   # -5
print(parse_number_eafp(" 7 "))  # 7
```

Риски EAFP

широкий try скрывает ошибки

исключения дороже проверок

не для всех операций есть исключение

часть действий необратима

Накладные расходы

	LBYL	EAFP
операция успешна	проверка if при каждом вызове	накладных расходов нет
операция неудачна	только проверка if	создание и перехват исключения

Эксперимент

```
def get_price_lbyl(prices: dict[str, int]) -> int:
    if "apple" in prices:
        return prices["apple"]
    return 0

def get_price_eafp(prices: dict[str, int]) -> int:
    try:
        return prices["apple"]
    except KeyError:
        return 0
```

Эксперимент

```
import timeit

present_prices = {"apple": 10}
missing_prices = {}


for title, prices in (
    ("key present", present_prices),
    ("key missing", missing_prices),
):
    for function in (get_price_lbyl, get_price_eafp):
        duration = timeit.timeit(
            lambda: function(prices), number=2_000_000
        )
        print(
            f"{title:12} {function.__name__:15} {duration:.3f} s"
        )
```

Типичный результат

	LBYL	EAFP
ключ есть	**0,125 с**	**0,102 с**
ключа нет	**0,091 с**	**0,386 с**

значения зависят от компьютера и версии Python

Когда что выбирать

EAFP	LBYL
неудача редка	неудача часта
внешние ресурсы: файлы, сеть	после проверки гонок нет
проверка сложна или неполна	проверка проста и дешева



try-finally

Блок finally

```
try:
    # основной код, при выполнении которого может возникнуть исключение
    ...

finally:
    # завершающие действия, выполняются всегда
    ...
```

finally

```
def run(mode: str) -> None:
    try:
        print("try: start")
        if mode == "error":
            raise ValueError("something went wrong")
        print("try: end")
    finally:
        print("finally: cleanup")

run("ok")
# try: start
# try: end
# finally: cleanup
```

finally при исключении

```
run("error")
# try: start
# finally: cleanup
# Traceback (most recent call last):
#   ...
# ValueError: something went wrong
```

finally и return

```
def get_value() -> int:
    try:
        return 1
    finally:
        print("cleanup")

print(get_value())
# cleanup
# 1
```

finally в цикле

Код

```
for number in range(3):
    try:
        if number == 1:
            continue

        print(f"body {number}")

    finally:
        print(f"finally {number}")
```

Вывод

```
body 0
finally 0
finally 1
body 2
finally 2
```

Освобождение ресурсов

число ресурсов ограничено

данные остаются в буфере

ресурс остается занятым

Утечка ресурса

```
file = open("data.txt", "w")
file.write("first line\n")

number = int("abc")  # ValueError

# не выполнится: файл останется открытым
file.write("second line\n")
file.close()
```

close() в finally

```
file = open("data.txt", "w")

try:
    file.write("first line\n")
    number = int("abc")  # ValueError
    file.write("second line\n")

finally:
    file.close()  # выполнится в любом случае
    print(f"File closed: {file.closed}")
# File closed: True
# Traceback (most recent call last):
#   ...
# ValueError: invalid literal for int() with base 10: 'abc'
```

open внутри try

```
def read_file(path: str) -> str:
    try:
        file = open(path)
        return file.read()

    finally:
        file.close()

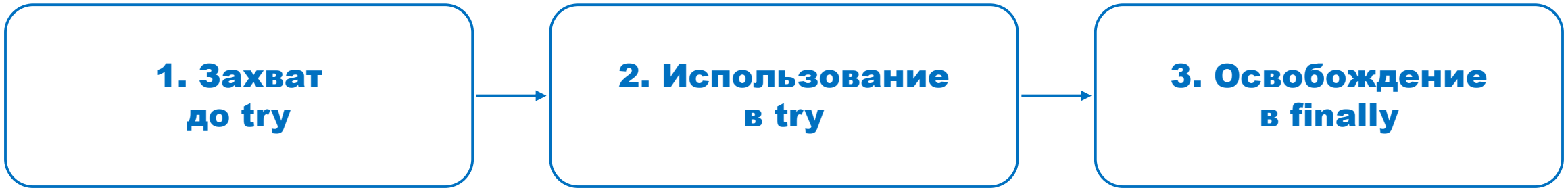
read_file("missing.txt")
# UnboundLocalError
```

Шаблон работы с ресурсами

```
resource = acquire()  # захват ресурса; при неудаче освобождать нечего

try:
    use(resource)     # использование

finally:
    release(resource)  # освобождение выполняется всегда
```



try-except-finally

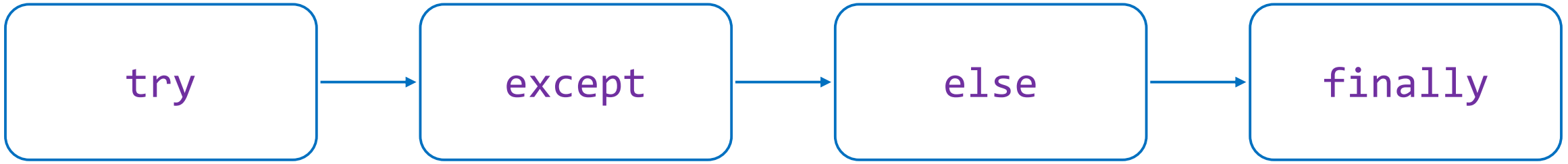
```
file = open("data.txt")

try:
    first_line = file.readline()
    value = int(first_line)

except ValueError:
    print("First line is not a number")
    value = 0

finally:
    file.close()
```

Полная форма try



основной код

реакция на ошибку

успешный исход

выполняется всегда

Исключение в finally

```
try:
    raise ValueError("original")

finally:
    # RuntimeError, ValueError в цепочке
    raise RuntimeError("from finally")
# ...
# ValueError: original
#
# During handling of the above exception, another exception occurred:
#
# ...
# RuntimeError: from finally
```

return и исключения

```
def get_result() -> str:
    try:
        raise ValueError("lost")
    finally:
        return "from finally"

print(get_result())
# SyntaxWarning: 'return' in a 'finally' block
# from finally
```

finally не выполняется

не выполняется	выполняется
аварийное завершение интерпретатора	`sys.exit()`
`os._exit()`	`KeyboardInterrupt`
остановка процесса системой	

Спасибо за внимание!